

ESTRUTURA DE DADOS

Árvore Binária de Busca (BST)

Material de Estudo com Simulado

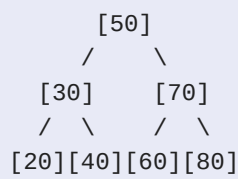
Contexto / Motivação

Arrays e listas encadeadas exigem $O(n)$ para busca. Uma BST mantém os dados ordenados por estrutura — cada nó garante que filhos à esquerda são menores e filhos à direita são maiores — permitindo busca, inserção e remoção em $O(\log n)$ no caso médio, sem o custo de reordenação de um array.

Dica: A BST é a base para estruturas mais avançadas (AVL, Red-Black Tree). Entender BST é pré-requisito para AVL — que aparece nas próximas AAs.

Propriedade da BST e Estrutura

Invariante: para todo nó N , todo valor na subárvore esquerda $< N.dado$, e todo valor na subárvore direita $> N.dado$. Essa invariante deve ser preservada em inserções e remoções.



Busca por 40: $40 < 50 \rightarrow \text{esq}$, $40 > 30 \rightarrow \text{dir}$, encontrado!

Figura 1 — BST com 7 nós. A busca por 40 percorre apenas 3 nós ($O(\log n)$ neste caso)

```
// AA10 - Árvore Binária de Busca (BST)
// Autor: Gabriel Gonçalves
// FATEC Ourinhos – Estrutura de Dados

#include <stdio.h>
#include <stdlib.h>

typedef struct No {
    int dado;
    struct No *esquerda;
    struct No *direita;
} No;

// Cria um novo nó
No* no_criar(int dado) {
    No *novo = malloc(sizeof(No));
    novo->dado = dado;
    novo->esquerda = NULL;
    novo->direita = NULL;
    return novo;
}
```

Código 1 — Estrutura do nó. Dois ponteiros filho em vez de um.

Operações: Inserção e Busca

Inserção — $O(\log n)$ médio, $O(n)$ pior caso

Percorre a árvore pela invariante até encontrar posição NULL. Retorna o ponteiro do nó atualizado — padrão recursivo clássico em BST.

```
No* arvore_inserir(No *raiz, int dado) {
    if (raiz == NULL)          // posição encontrada
        return no_criar(dado);
    if (dado < raiz->dado)
        raiz->esquerda = arvore_inserir(raiz->esquerda, dado);
    else if (dado > raiz->dado)
        raiz->direita = arvore_inserir(raiz->direita, dado);
    // dado == raiz->dado: ignora duplicata
    return raiz;
}

No* arvore_buscar(No *raiz, int dado) {
    if (raiz == NULL || raiz->dado == dado)
        return raiz;          // NULL = não encontrado
    if (dado < raiz->dado)
        return arvore_buscar(raiz->esquerda, dado);
    return arvore_buscar(raiz->direita, dado);
}
```

Código 2 — Inserção e busca recursivas. Notar que inserção retorna raiz (pode mudar se NULL).

Operações: Remoção

Remoção é a operação mais complexa da BST. Três casos distintos devem ser tratados:

- Nó folha (sem filhos): simplesmente libera o nó e retorna NULL
- Nó com 1 filho: substitui o nó pelo filho sobrevivente
- Nó com 2 filhos: encontra o SUCESSOR IN-ORDER (menor da subárvore direita), copia seu dado, remove o sucessor recursivamente

```
// Encontra o menor nó (mais à esquerda)
No* arvore_minimo(No *raiz) {
    while (raiz->esquerda != NULL)
        raiz = raiz->esquerda;
    return raiz;
}

No* arvore_remove(No *raiz, int dado) {
    if (raiz == NULL) return NULL;

    if (dado < raiz->dado)
        raiz->esquerda = arvore_remove(raiz->esquerda, dado);
    else if (dado > raiz->dado)
        raiz->direita = arvore_remove(raiz->direita, dado);
    else {
        // Caso 1 e 2: 0 ou 1 filho
        if (raiz->esquerda == NULL) {
            No *temp = raiz->direita;
            free(raiz);
            return temp;
        }
        if (raiz->direita == NULL) {
            No *temp = raiz->esquerda;
            free(raiz);
            return temp;
        }
        // Caso 3: 2 filhos – substituir pelo sucessor
        No *suc = arvore_minimo(raiz->direita);
        raiz->dado = suc->dado;
        raiz->direita = arvore_remove(raiz->direita, suc->dado);
    }
    return raiz;
}
```

Código 3 — Remoção com 3 casos. O sucessor in-order preserva a invariante da BST.

Travessias

As três travessias clássicas diferem na ordem de visita do nó raiz em relação aos filhos:

Travessia	Ordem	Uso típico
In-order (esq→raiz→dir)	Crescente	Listar elementos ordenados
Pre-order (raiz→esq→dir)	Raiz primeiro	Serializar/copiar a árvore
Post-order (esq→dir→raiz)	Raiz por último	Destruir a árvore (free)

```
void arvore_in_order(No *raiz) {
    if (raiz == NULL) return;
    arvore_in_order(raiz->esquerda);
    printf("%d ", raiz->dado); // visita raiz no meio
    arvore_in_order(raiz->direita);
}

void arvore_destruir(No *raiz) {
    if (raiz == NULL) return;
    arvore_destruir(raiz->esquerda); // libera filhos
    arvore_destruir(raiz->direita);  // antes da raiz
    free(raiz);                      // post-order!
}
```

Código 4 — In-order produz saída ordenada. Destruição usa post-order por segurança.

Complexidade e Trade-offs

Operação	Caso Médio	Pior Caso	Observação
Inserir	$O(\log n)$	$O(n)$	Pior caso: inserir em ordem crescente
Buscar	$O(\log n)$	$O(n)$	Árvore degenerada perde toda vantagem
Remover	$O(\log n)$	$O(n)$	Complexidade do árv. mínimo inclusa
In-order	$O(n)$	$O(n)$	Visita todos os nós — inevitável
Destruir	$O(n)$	$O(n)$	Todo nó precisa ser liberado

Dica: Se a entrada for ordenada (ex: 1,2,3,4,5), a BST degenera numa lista ligada — todas as operações viram $O(n)$. A AVL resolve isso com rebalanceamento automático, mantendo $O(\log n)$ garantido.

SIMULADO

Questão 1 (Conceitual)

Qual a invariante (propriedade fundamental) que uma BST deve sempre manter? Por que essa propriedade torna a busca mais eficiente do que numa lista encadeada?

Questão 2 (Raciocínio)

Trace a inserção sequencial dos valores [50, 30, 70, 20, 40] numa BST inicialmente vazia. Desenhe a árvore resultante e indique a raiz e as folhas.

Questão 3 (Conceitual)

Explique os 3 casos da remoção em BST. Qual caso é mais complexo e por quê? Qual estratégia é usada para preservar a invariante no caso mais complexo?

Questão 4 (Código)

Complete a função abaixo que verifica se um valor existe na BST. A função deve retornar 1 se encontrado, 0 caso contrário. `int arvore_contem(No *raiz, int dado) { if (raiz == NULL) return _____; if (dado == raiz->dado) return _____; if (dado < raiz->dado) return arvore_contem(_____, dado); return arvore_contem(_____, dado); }`

// Escreva sua resposta aqui:

Questão 5 (Conceitual)

Por que a travessia in-order de uma BST produz os elementos em ordem crescente?
Que tipo de problema você resolveria usando a travessia post-order?

Questão 6 (Raciocínio)

Dado o seguinte código C, qual será a saída para a BST com valores [50, 30, 70, 20, 40]?
`void f(No *r) { if (!r) return; f(r->direita); printf("%d ", r->dado); f(r->esquerda); }`
A) 20 30 40 50 70 B) 70 50 40 30 20 C) 50 30 20 40 70 D) 20 40 30 70 50

GABARITO COMENTADO

Questão 1 (Conceitual):

Invariante: todo nó N satisfaz $\text{esquerda} < N < \text{direita}$ em toda subárvore. Isso permite busca binária: a cada nível, descartamos metade da árvore. Em lista encadeada não há ordenação estrutural, logo busca é sempre $O(n)$. Na BST o caso médio é $O(\log n)$.

Questão 2 (Raciocínio):

Raiz: 50. Filhos: 30 (esq), 70 (dir). Filhos de 30: 20 (esq), 40 (dir). 70 é folha. Folhas: 20, 40, 70. Altura = 3 (raiz no nível 1).

Questão 3 (Conceitual):

Caso 1: folha — `free()` direto. Caso 2: 1 filho — substituir pelo filho. Caso 3: 2 filhos — o mais complexo, pois remover o nó diretamente quebraria a árvore. Solução: encontrar o sucessor in-order (menor da subárvore direita), copiar seu dado para o nó atual, e remover o sucessor recursivamente. Isso preserva a invariante.

Questão 4 (Código):

`return 0; / return 1; / raiz->esquerda / raiz->direita.` Lógica: NULL = não encontrado = 0; igual = encontrado = 1; menor = desce esquerda; maior = desce direita.

Questão 5 (Conceitual):

In-order visita esquerda→raiz→direita. Como esquerda < raiz < direita (invariante), os elementos são visitados em ordem crescente. Post-order (esquerda→direita→raiz) é ideal para destruição da árvore: garante que os filhos são liberados antes do pai, evitando ponteiros soltos.

Questão 6 (Raciocínio):

B) 70 50 40 30 20. A função é uma travessia in-order invertida (direita→raiz→esquerda), produzindo os elementos em ordem DECRESCENTE. Para a BST [50,30,70,20,40]: 70, 50, 40, 30, 20.